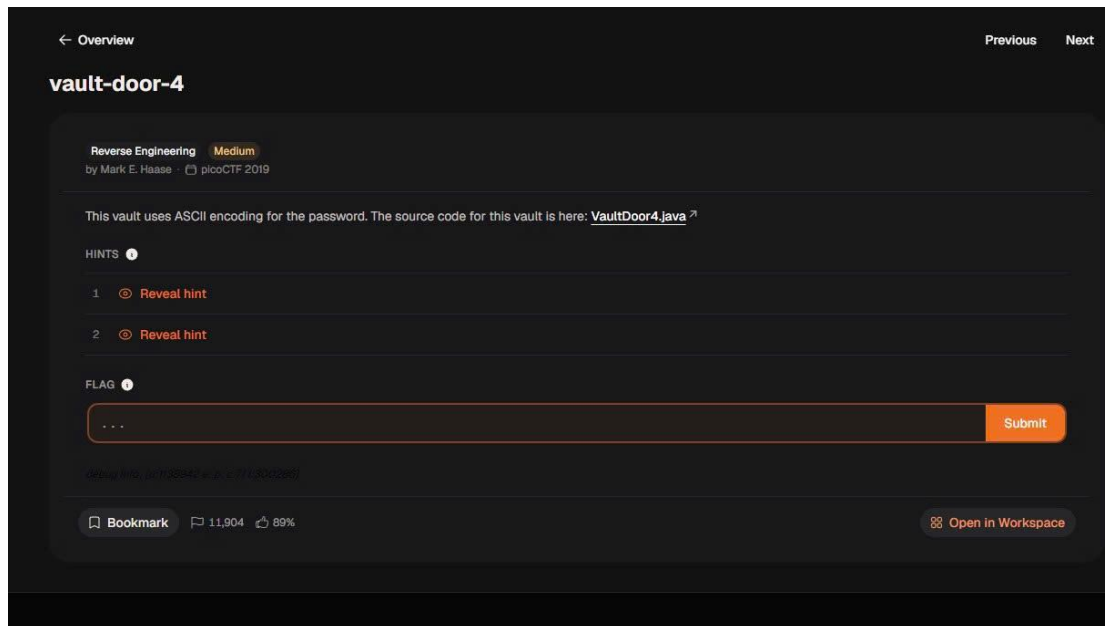


Vault Door Series - picoCTF Write-up (Reverse Engineering)



Thử thách này thiên về: Reverse Engineering -> đọc ngược lại mã của file

Đề bài đã cho:

- VaultDoor4.java (file java)
- ASCII được sử dụng để encode flag trong bài
- Vì đề bài không cho flag format sẵn nên có thể đã bị giấu trong file

Goal: Đọc file java --> kiểm byte/ciphertext bị giấu --> giải mã lấy flag

I. Tải và đọc file

- Có thể click tải file về và sử dụng trong Terminal hoặc dùng lệnh `wget` đi với link address của file để lưu trong webshell

- Sau đó execute file java bằng lệnh “ java VaultDoor4 ”, và nó hiện lên “Cần nhập mật khẩu của Vault”

-Vì chưa có mật khẩu nên cần phải kiếm 1 hướng đi khác, lúc này hãy đọc cấu trúc của file bằng lệnh “cat”.

- Toàn bộ dòng lệnh của file đã được xuất ra trong webshell, trong đó có 2 thứ đáng lưu ý:

+ Đầu tiên là:

```
System.out.print("Enter vault password: ");
String userInput = scanner.next();
String input = userInput.substring("academy{".length(),userInput.length()-1);
if (vaultDoor.checkPassword(input)) {
    System.out.println("Access granted.");
} else {
    System.out.println("Access denied!");
}
```

- Đoạn mã xử lý chuỗi đầu vào thông qua dòng lệnh:

“ String input = userInput.substring("academy{".length(),
userInput.length()-1); ”

- Vị trí bắt đầu ("academy{".length()): Hàm này trả về giá trị là 8 (độ dài của chuỗi "academy{"). Bằng cách bắt đầu cắt từ vị trí index 8, chương trình sẽ bỏ qua 8 ký tự đầu tiên. Điều này ngầm định rằng flag hợp lệ phải được bắt đầu bằng chuỗi academy{.

- Và nếu password có định dạng academy{..} là đúng thì nó sẽ là flag của bài

+ Cái thứ hai cần chú ý là:

```
public boolean checkPassword(String password) {
    byte[] passBytes = password.getBytes();
    byte[] myBytes = {
        106 , 85 , 53 , 116 , 95 , 52 , 95 , 98 ,
        0x55, 0x6e, 0x43, 0x68, 0x5f, 0x30, 0x66, 0x5f,
        0142, 0131, 0164, 063 , 0163, 0137, 066 , 0141,
        '8' , 'a' , '9' , 'e' , '9' , '6' , 'b' , 'c' ,
    };
}
```

- Nhìn vào dòng code trên, có thể suy luận ra rằng phần password bị mã hóa sẽ nằm ở trong các Bytes này.

Tổng có 4 dòng Bytes, quan sát lần lượt:

- + Dòng thứ nhất sẽ là định dạng Decimal
- + Dòng thứ hai sẽ là định dạng Hexadecimal (Bắt đầu = 0x)
- + Dòng thứ ba sẽ là định dạng Octal (Bắt đầu số 0)
- + Dòng thứ tư sẽ là định dạng Character/ASCII (Ký tự) vì được đặt trong dấu ngoặc đơn.

- Từ đó có thể suy ra những Bytes này được giải mã và ghép lại sẽ tạo thành password/flag của bài.

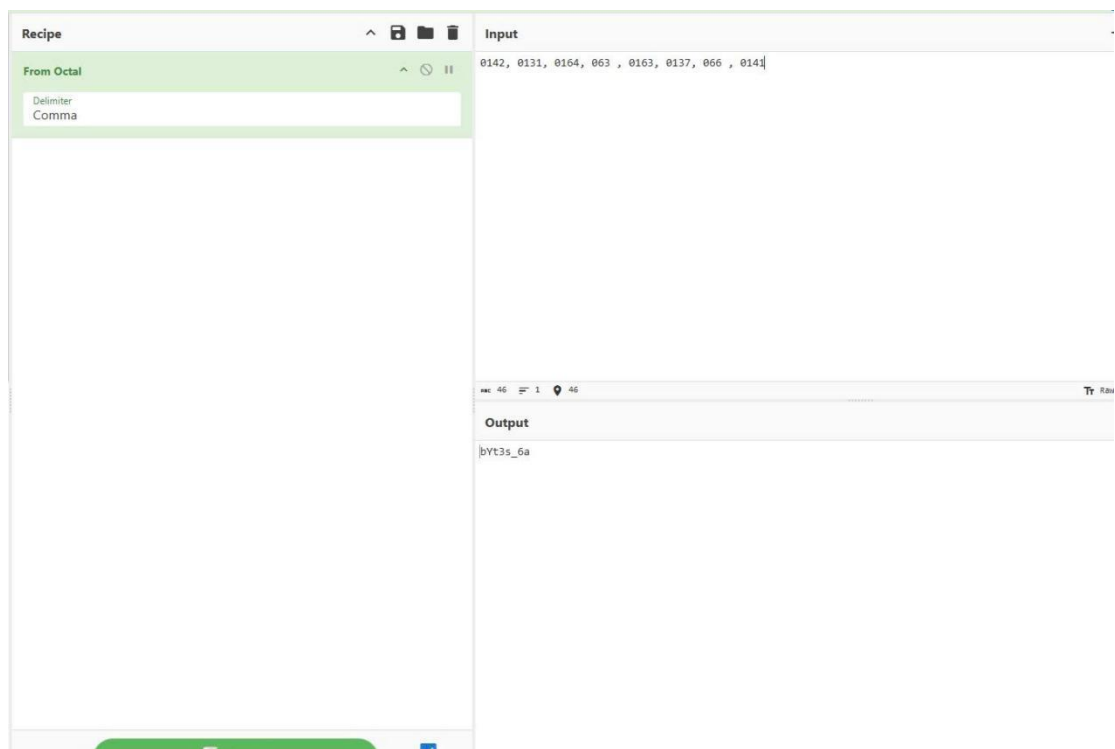
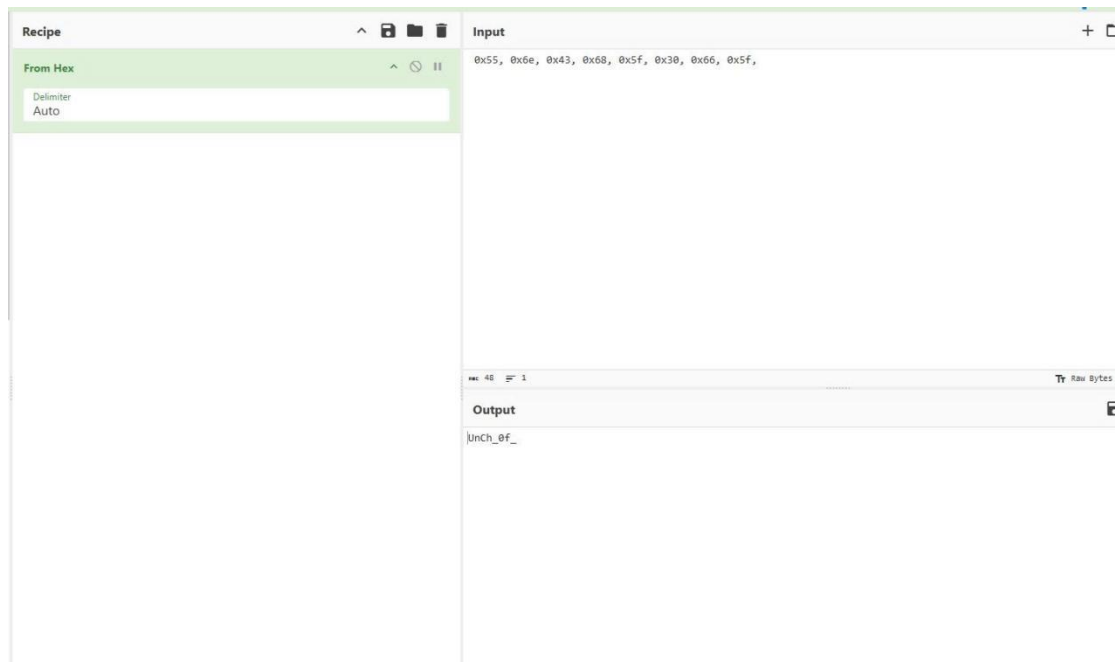
II. Decode Bytes.

- Lên CyberChef để giải lần lượt từng dòng Bytes.

The screenshot shows the CyberChef web application. The 'Recipe' panel on the left has a 'From Decimal' recipe selected. The 'Delimiter' is set to 'Space'. The 'Support signed values' checkbox is unchecked. The 'Input' panel on the right contains the text: '106 , 85 , 53 , 116 , 95 , 52 , 95 , 98 ,'. The 'Output' panel on the right shows an error message: 'Data is not a valid byteArray: [106,null,85,null,53,n...'.

- Phải chuyển dòng Delimiter thành Comma để tránh bị lỗi như trên:

The screenshot shows the CyberChef web application. The 'Recipe' panel on the left has a 'From Decimal' recipe selected. The 'Delimiter' is set to 'Comma'. The 'Support signed values' checkbox is unchecked. The 'Input' panel on the right contains the text: '106 , 85 , 53 , 116 , 95 , 52 , 95 , 98 ,'. The 'Output' panel on the right shows the decoded text: 'just_4_b'.



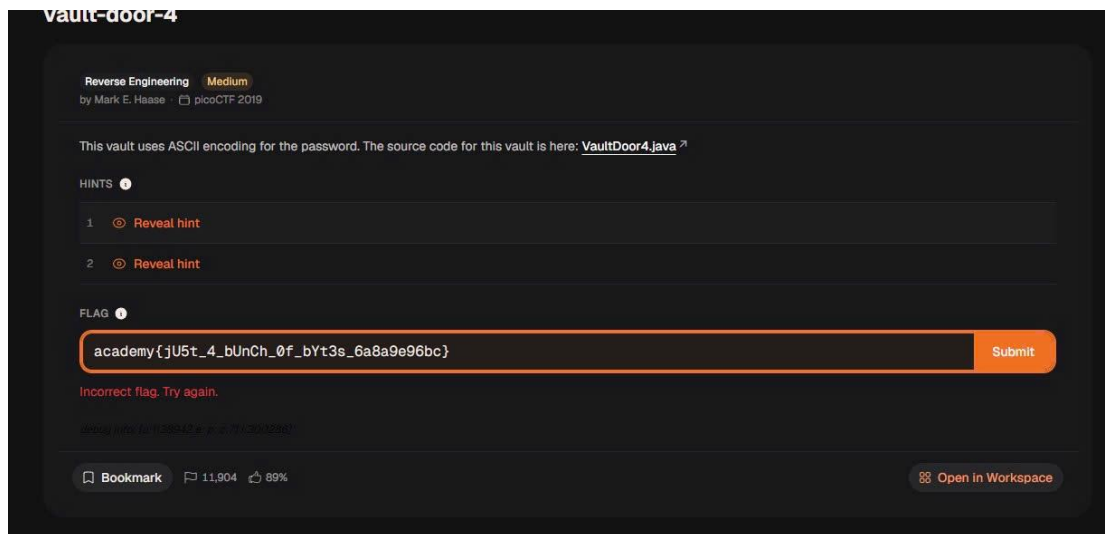
Và dòng cuối cùng chỉ cần ghép lại từng chữ, sau đó ta sẽ có được 1 password:

academy{fjU5t_4_bUnCh_of_bYt3s_6a8a9e96bc}

- Sau đó nhập thử password vào file java đã được thực thi:

```
} BaoPhucUIT12112008-academy@webshell:~$ java VaultDoor4
Enter vault password: academy{jU5t_4_bUnCh_0f_bYt3s_6a8a9e96bc}
Access granted.
BaoPhucUIT12112008-academy@webshell:~$
```

Dòng “Access granted” cho thấy rằng password đã đúng
---> Đây là flag của bài.



- Nhập flag vào và trang sẽ hiển thị kết quả:

